

Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря
Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №1

**з дисципліни «Об'єктно-орієнтоване
програмування»**

на тему

**«Знайомство із середовищем розробки програм
Microsoft Visual Studio та складання модульних
проєктів програм на C++»**

Виконав:

Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

Завдання

1. Створити у середовищі MS Visual Studio C++ проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налогодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту

Номер у списку групи: **Ж = 16**

- **B1 = Ж mod 4 = 16 mod 4 = 0** — вікно діалогу для вводу тексту з рядком вводу (Edit Control) та двома кнопками: [Так] і [Відміна]. При натисканні [Так] введений текст відображається у головному вікні.
- **B2 = (Ж + 1) mod 4 = 17 mod 4 = 1** — вікно діалогу з повзунком горизонтального скролінгу (Horizontal Scroll Bar) та двома кнопками: [Так] і [Відміна]. Користувач вводить число від 1 до 100, після натискання [Так] число відображається у головному вікні.

Кожен модуль реалізовано в окремих файлах `.cpp` / `.h` / `.rc` з прихованими callback-функціями (`static`) та власними ресурсами. Інтерфейс модуля — одна функція `extern int Func_MODx(...)` .

Вихідний текст програми

Головний файл Lab1.cpp

```
#include <windows.h>
#include <tchar.h>
```

```

#include "resource.h"
#include "module1.h"
#include "module2.h"

#define MAX_TEXT 256

static TCHAR szWindowClass[] = _T("Lab1WindowClass");
static TCHAR szTitle[] = _T("Lab 1 – Stepanenko Denys, IM-051");

static TCHAR g_text[MAX_TEXT] = _T("");
static int g_number = 0;
static bool g_showText = false;
static bool g_showNumber = false;

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
        case WM_COMMAND:
        {
            int wmId = LOWORD(wParam);
            switch (wmId)
            {
                case ID_WORK1:
                {
                    TCHAR buf[MAX_TEXT];
                    if (Func_MOD1(hWnd, buf, MAX_TEXT) == 1)
                    {
                        _tcsncpy_s(g_text, MAX_TEXT, buf, _TRUNCATE);
                        g_showText = true;
                        g_showNumber = false;
                        InvalidateRect(hWnd, NULL, TRUE);
                    }
                }
            }
            break;

            case ID_WORK2:
            {
                int val;
                if (Func_MOD2(hWnd, &val) == 1)
                {
                    g_number = val;
                    g_showNumber = true;
                    g_showText = false;
                    InvalidateRect(hWnd, NULL, TRUE);
                }
            }
            break;

            case IDM_EXIT:
                DestroyWindow(hWnd);
                break;
        }
    }
}

```

```

        case IDM_ABOUT:
            MessageBox(hWnd,
                _T("Lab 1 – Modular Win32 Project\nStepanenko Denys, IM-051\nVariant: B1=0,
B2=1"),
                _T("About"), MB_OK | MB_ICONINFORMATION);
            break;

        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
    }
}
break;

case WM_PAINT:
{
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);

    if (g_showText)
    {
        TextOut(hdc, 20, 30, _T("Entered text:"), 13);
        TextOut(hdc, 20, 55, g_text, (int)_tcslen(g_text));
    }
    else if (g_showNumber)
    {
        TCHAR buf[32];
        _stprintf_s(buf, 32, _T("Selected number: %d"), g_number);
        TextOut(hdc, 20, 30, buf, (int)_tcslen(buf));
    }
    else
    {
        TextOut(hdc, 20, 30, _T("Select Work -> Work 1 or Work 2 from menu"), 43);
    }

    EndPaint(hWnd, &ps);
}
break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

int APIENTRY _tWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPTSTR lpCmdLine, int nCmdShow)
{

```

```

UNREFERENCED_PARAMETER(hPrevInstance);
UNREFERENCED_PARAMETER(lpCmdLine);

WNDCLASSEX wcex;
wcex.cbSize = sizeof(WNDCLASSEX);
wcex.style = CS_HREDRAW | CS_VREDRAW;
wcex.lpfnWndProc = WndProc;
wcex.cbClsExtra = 0;
wcex.cbWndExtra = 0;
wcex.hInstance = hInstance;
wcex.hIcon = LoadIcon(NULL, IDI_APPLICATION);
wcex.hCursor = LoadCursor(NULL, IDC_ARROW);
wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
wcex.lpszMenuName = MAKEINTRESOURCE(IDR_MAINMENU);
wcex.lpszClassName = szWindowClass;
wcex.hIconSm = LoadIcon(NULL, IDI_APPLICATION);

if (!RegisterClassEx(&wcex))
{
    MessageBox(NULL, _T("Failed to register window class"),
        _T("Error"), MB_OK | MB_ICONERROR);
    return 1;
}

HWND hWnd = CreateWindow(
    szWindowClass, szTitle,
    WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, CW_USEDEFAULT,
    500, 350,
    NULL, NULL, hInstance, NULL);

if (!hWnd)
{
    MessageBox(NULL, _T("Failed to create window"),
        _T("Error"), MB_OK | MB_ICONERROR);
    return 1;
}

ShowWindow(hWnd, nCmdShow);
UpdateWindow(hWnd);

MSG msg;
while (GetMessage(&msg, NULL, 0, 0))
{
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}

return (int)msg.wParam;
}

```

Модуль 1 — module1.h

```
#pragma once
#include <windows.h>

extern int Func_MOD1(HWND hWnd, TCHAR* outText, int maxLen);
```

Модуль 1 — module1.cpp

```
#include <windows.h>
#include <tchar.h>
#include "module1.h"
#include "module1_resource.h"

static INT_PTR CALLBACK DlgProc1(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
        case WM_INITDIALOG:
            SetDlgItemText(hDlg, IDC_EDIT_TEXT, _T(""));
            return TRUE;

        case WM_COMMAND:
        {
            int wmId = LOWORD(wParam);
            switch (wmId)
            {
                case IDC_BTN_OK:
                {
                    TCHAR buf[MAX_PATH];
                    GetDlgItemText(hDlg, IDC_EDIT_TEXT, buf, MAX_PATH);
                    static TCHAR s_result[MAX_PATH];
                    _tcsncpy_s(s_result, MAX_PATH, buf, _TRUNCATE);
                    EndDialog(hDlg, (INT_PTR)s_result);
                    return TRUE;
                }

                case IDC_BTN_CANCEL:
                    EndDialog(hDlg, 0);
                    return TRUE;
            }
        }
        break;

        case WM_CLOSE:
            EndDialog(hDlg, 0);
            break;

        default:
            return FALSE;
    }
}
```

```

        return FALSE;
    }

    int Func_MOD1(HWND hWnd, TCHAR* outText, int maxlen)
    {
        INT_PTR result = DialogBox(
            (HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE),
            MAKEINTRESOURCE(IDD_DIALOG1),
            hWnd, DlgProc1);

        if (result == 0)
            return 0;

        TCHAR* text = (TCHAR*)result;
        _tcsncpy_s(outText, maxlen, text, _TRUNCATE);
        return 1;
    }

```

Модуль 1 — module1.rc

```

#include "module1_resource.h"

IDD_DIALOG1 DIALOGEX 0, 0, 280, 120
STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "Work 1 – Text Input"
FONT 8, "MS Shell Dlg", 400, 0, 0x1
BEGIN
    LTEXT        "Enter text:", -1, 10, 10, 100, 12
    EDITTEXT     IDC_EDIT_TEXT, 10, 25, 260, 14, ES_AUTOHSCROLL
    PUSHBUTTON   "OK", IDC_BTN_OK, 160, 95, 50, 14
    PUSHBUTTON   "Cancel", IDC_BTN_CANCEL, 220, 95, 50, 14
END

```

Модуль 2 — module2.h

```

#pragma once
#include <windows.h>

extern int Func_MOD2(HWND hWnd, int* outValue);

```

Модуль 2 — module2.cpp

```

#include <windows.h>
#include <tchar.h>
#include "module2.h"
#include "module2_resource.h"

#define MIN_VAL 1

```

```

#define MAX_VAL 100

static int s_currentValue = 50;

static void UpdateValueDisplay(HWND hDlg)
{
    TCHAR buf[16];
    _stprintf_s(buf, 16, _T("%d"), s_currentValue);
    SetDlgItemText(hDlg, IDC_SCROLL_VALUE, buf);
}

static INT_PTR CALLBACK DlgProc2(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
    case WM_INITDIALOG:
        s_currentValue = 50;
        HWND hScroll;
        hScroll = GetDlgItem(hDlg, IDC_SCROLLBAR);
        SetScrollRange(hScroll, SB_CTL, MIN_VAL, MAX_VAL, FALSE);
        SetScrollPos(hScroll, SB_CTL, s_currentValue, TRUE);
        UpdateValueDisplay(hDlg);
        return TRUE;

    case WM_HSCROLL:
    {
        HWND hScroll = (HWND)lParam;
        int scrollCode = LOWORD(wParam);
        int pos = s_currentValue;

        switch (scrollCode)
        {
        case SB_LINELEFT: pos--; break;
        case SB_LINERIGHT: pos++; break;
        case SB_PAGELEFT: pos -= 10; break;
        case SB_PAGERIGHT: pos += 10; break;
        case SB_THUMBTRACK:
        case SB_THUMBPOSITION:
            pos = HIWORD(wParam); break;
        }

        if (pos < MIN_VAL) pos = MIN_VAL;
        if (pos > MAX_VAL) pos = MAX_VAL;

        if (pos != s_currentValue)
        {
            s_currentValue = pos;
            SetScrollPos(hScroll, SB_CTL, s_currentValue, TRUE);
            UpdateValueDisplay(hDlg);
        }
    }
    break;
}

```



```

case WM_COMMAND:
{
    int wmId = LOWORD(wParam);
    switch (wmId)
    {
        case IDC_BTN_OK:
            EndDialog(hDlg, s_currentValue);
            return TRUE;
        case IDC_BTN_CANCEL:
            EndDialog(hDlg, -1);
            return TRUE;
    }
}
break;

case WM_CLOSE:
    EndDialog(hDlg, -1);
    break;

default:
    return FALSE;
}
return FALSE;
}

int Func_MOD2(HWND hWnd, int* outValue)
{
    INT_PTR result = DialogBox(
        (HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE),
        MAKEINTRESOURCE(IDD_DIALOG2),
        hWnd, DlgProc2);

    if (result == -1)
        return 0;

    *outValue = (int)result;
    return 1;
}

```

Модуль 2 — module2.rc

```

#include "module2_resource.h"

IDD_DIALOG2 DIALOGEX 0, 0, 280, 140
STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "Work 2 – Scrollbar Input (1-100)"
FONT 8, "MS Shell Dlg", 400, 0, 0x1
BEGIN
    LTEXT                "Drag the scrollbar to select a number (1-100):", -1, 10, 10, 260, 12
    SCROLLBAR             IDC_SCROLLBAR, 10, 30, 260, 15, SBS_HORZ

```

```
CTEXT          "50", IDC_SCROLL_VALUE, 10, 55, 260, 14
PUSHBUTTON     "OK", IDC_BTN_OK, 160, 115, 50, 14
PUSHBUTTON     "Cancel", IDC_BTN_CANCEL, 220, 115, 50, 14
END
```

Головний ресурс — Lab1.rc

```
#include "resource.h"

IDR_MAINMENU MENU
BEGIN
    POPUP "&File"
    BEGIN
        MENUITEM "E&xit", IDM_EXIT
    END
    POPUP "&Work"
    BEGIN
        MENUITEM "Work &1", ID_WORK1
        MENUITEM "Work &2", ID_WORK2
    END
    POPUP "&Help"
    BEGIN
        MENUITEM "&About", IDM_ABOUT
    END
END
```

Діаграми

Діаграма залежностей файлів та модулів

```
Lab1.cpp
├─ <windows.h>
├─ <tchar.h>
├─ resource.h
├─ module1.h
│   └─ <windows.h>
└─ module2.h
    └─ <windows.h>

module1.cpp
├─ <windows.h>
├─ <tchar.h>
├─ module1.h
└─ module1_resource.h

module2.cpp
├─ <windows.h>
├─ <tchar.h>
└─ module2.h
```

```
└─ module2_resource.h

Lab1.rc
└─ resource.h

module1.rc
└─ module1_resource.h

module2.rc
└─ module2_resource.h
```

Перехресних `#include` між модулями немає — кожен модуль незалежний.

Скріншоти

Головне вікно програми



Рис. 1. Головне вікно програми з меню *File*, *Work*, *Help*

Вікно діалогу «Робота1»



Рис. 2. Вікно діалогу модуля 1 — введення тексту (варіант $B1=0$)

Результат «Робота1» у головному вікні



Рис. 3. Виведення введеного тексту у головному вікні

Вікно діалогу «Робота2»



Рис. 4. Вікно діалогу модуля 2 — повзунок скролінгу (варіант $B2=1$)

Результат «Робота2» у головному вікні



Рис. 5. Виведення вибраного числа у головному вікні

Висновки

У лабораторній роботі я виконав завдання згідно свого варіанту (Ж = 16). Створено модульний проєкт Lab1, що складається з головного файлу та двох незалежних модулів. Модуль 1 реалізує вікно діалогу для вводу тексту (варіант В1 = 0), модуль 2 — вікно діалогу з горизонтальним повзунком скролінгу для вводу числа від 1 до 100 (варіант В2 = 1).

Кожен модуль має власний інтерфейс (одна функція `extern int Func_MODx(...)`), приховану callback-функцію (`static`), власний файл ресурсів `.rc` та власний заголовок ресурсів. Перехресних `#include` - зв'язків між модулями немає, що забезпечує їхню повну незалежність.

Лабораторна робота дала практичні навички роботи з Windows API: реєстрація класу вікна, створення вікна, обробка повідомлень (`WM_COMMAND`, `WM_PAINT`, `WM_HSCROLL`), використання діалогових вікон (`DialogBox`), робота з елементами управління (`Edit Control`, `Scroll Bar`), виведення тексту (`TextOut`).